Interim Architecture Report: Automaton-Auditor

Project: Forensic Governance Swarm for AI-Generated Code

Status: Phases 1–2 Complete (Detective Layer)

Author: Eyor Getachew

Date: Wednesday, February 25, 2026

Tenacious Intelligence Corporation

1. Architectural Decision Rationale

1.1 State Management: Pydantic & TypedDict vs. Plain Dicts

I implemented a strictly typed AgentState using TypedDict and Annotated Pydantic models with custom reducers (**operator.add**, **operator.ior**).

- **Alternative Considered:** Plain Python Dictionaries.
- **Why Rejected:** Plain dictionaries (dicts.) allow for **"Silent State Corruption."** In a parallel swarm, if one agent introduces a typo in a key (e.g., ”evidnce” instead of evidence), the graph will not fail immediately but will cause downstream judges to hallucinate based on missing data.
- **Failure Mode Prevented:** Pydantic ensures runtime validation. By using Annotated list reducers, I prevent race conditions during the Fan-In phase, ensuring evidence from three parallel detectives is merged rather than overwritten.
- **Trade-off consideration:** While Pydantic validation guarantees strict runtime type safety, it incurs some performance overhead compared to plain dictionaries, especially during high-frequency state updates. This trade-off is acceptable given the critical importance of data integrity in the audit swarm environment.

1.2 Code Analysis: AST Parsing vs. Regex

I utilized the ast module to traverse the Abstract Syntax Tree of the target repository.

- **Alternative Considered:** Regular Expressions (Regex).
- **Why Rejected:** Regex is **grammatically unaware**. It fails on multiline function calls, nested structures, or aliased imports (e.g., import langgraph as lg). AST allows the **RepoInvestigator** to deterministically verify the *logic* of the graph wiring (searching for **add_edge** calls with list arguments) regardless of formatting.
- **Failure Mode Prevented:** Prevents "False Negatives" where valid parallel code is missed due to non-standard indentation or commenting.

- **Trade-off consideration:** AST parsing provides robust handling of complex nested structures but introduces additional implementation complexity and computational overhead. Regex is simpler and faster but unreliable for nuanced code analysis, risking missed or incorrect results in this forensic context.

1.3 Sandboxing: Temp-Directory Isolation

Decision: Every audit triggers a **clone_repo_sandboxed** command using **tempfile.mkdtemp().**

- **Alternative Considered:** Cloning into a persistent /data folder.
- **Why Rejected:** Persistent folders create **Workspace Pollution**. If two audits run concurrently, they could cross-contaminate codebases.
- **Failure Mode Prevented:** Ensures that the "**Evidence**" gathered is 100% unique to the specific commit being audited, preventing "Forensic Bleed" between audit sessions.

1.4 Ingestion: RAG-Lite via Docling

Decision: Direct conversion of PDF to Markdown for context-window ingestion.

- **Trade-off:** We skipped a vector database (standard RAG) to save on latency. Since the audit reports are relatively small, providing the full Markdown text to the LLM ensures it doesn't "miss" nuances that a similarity search might skip.

2. StateGraph Architecture Diagram

2.1 Current Implementation (Detectives)

The current flow utilizes a **Triple Fan-Out** pattern.

- **START ➔ Detectives:** Parallel execution of **repo_investigator, doc_analyst**, and **vision_inspector.**
- **Data Type:** Each detective emits a List[Evidence].
- **Aggregation:** The evidence_aggregator node performs a **Metacognitive Sync**, identifying discrepancies between code facts and document claims.

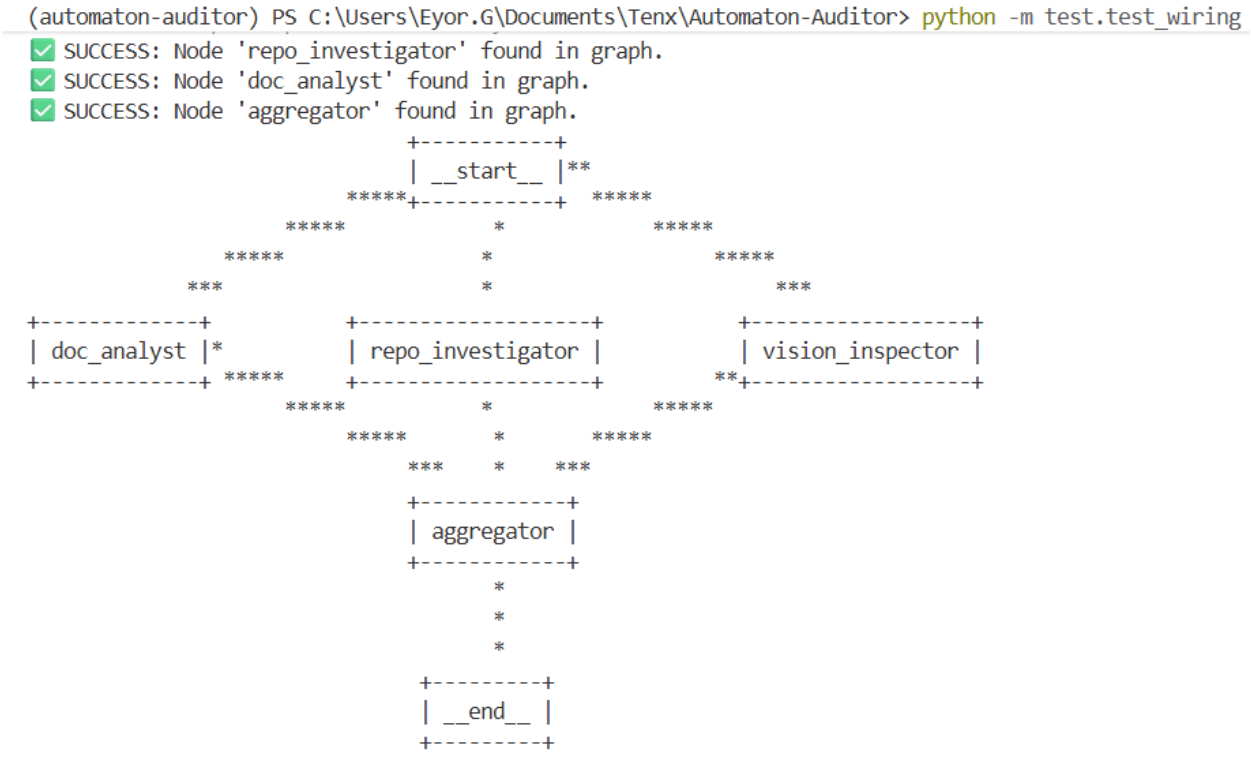


Fig 1. The Detective Fan-Out (The First Diamond)

The process workflow looks like this:

- The **__start__** is the entry point of the workflow, and this is where the execution begins.
- After the execution, the parallel processing starts working. The **doc_analyst** analyzes the PDF format documentation. The **repo_investigator** inspects repository-related things. The **vision_inspector** handles visual analysis like

image, such as images and diagrams. These run independently but are part of the same pipeline.

- The **aggregator** node collects outputs from previous nodes. It merges results, combines insights, and resolves conflicts between agent findings.
- The **__end__** is the final state of the workflow. Execution stops here.

The architecture is a Multi-Agent Orchestration pattern, which is common in LLM agent frameworks and LangGraph.

Tenacious Intelligence Corporation

3. Gap Analysis & Forward Plan

3.1 Known Gaps

- **Persona Divergence:** Currently, I have the evidence, but the "**Judges**" have not been prompted with distinct legal/technical personalities.
- **Conflict Resolution:** The "**Chief Justice**" logic for handling a 1/5 score from the Prosecutor vs. a 5/5 from the Defense is currently a pass-through stub.

3.2 Concrete Plan for Phases 3 &4

Phase	Task	Concrete Implementation	Risk Mitigation
3.1	**Judicial Fan-Out**	Implement three parallel nodes: Prosecutor, Defense, and TechLead. Each will use .with_structured_output(JudicialOpinion).	Prevents "Opinion Drift" where LLMs ignore the scoring rubric.
3.2	**Persona Prompting**	Use distinct system prompts (Prosecutor focuses on AST gaps; Defense focuses on intent).	Prevents "Groupthink" where judges converge on the same score without debate.
4	**Synthesis Engine**	The ChiefJustice node will apply a **Weighted Variance Rule**: If score variance > 2, it triggers a "Dissent Summary" requirement.	Prevents the system from averaging scores and hiding critical failures.

3.3 The Synthesis Engine: Algorithmic Reconciliation

The "**Chief Justice**" node is not a simple summarizer; it is a **Deterministic Synthesis Engine** designed to solve the "**Averaging Bias**" common in multi-agent systems.

Logic Gate 1: Weighted Variance Detection

Instead of a simple mean, the engine calculates the variance σ^2 between the Prosecutor, Defense, and Tech Lead scores.

- **If Variance ≤ 1:** The system assumes a "Judicial Consensus" and proceeds to final report generation.
- **If Variance > 2:** The system triggers a **Conflict Protocol**. The Chief Justice is forced to generate a **Dissent Summary**, explicitly quoting the evidence that caused the deadlock (e.g., "The Prosecutor cites the absence of parallel edge logic, while the Defense cites the intent shown in the commit history").

Logic Gate 2: The "Tech Lead" Override

In cases of a tie or extreme divergence, the Tech Lead's score—backed by the **AST Evidence**—carries a 1.5x weight. This ensures that "vibe-based" arguments from the Defense or Prosecutor cannot overrule hard engineering facts found in the code. The final formula looks like this:

$$\mathbf{FinalScore} = \frac{P + D + (1.5 \times T)}{3.5}$$

Failure Mode Mitigation: "Groupthink" Prevention

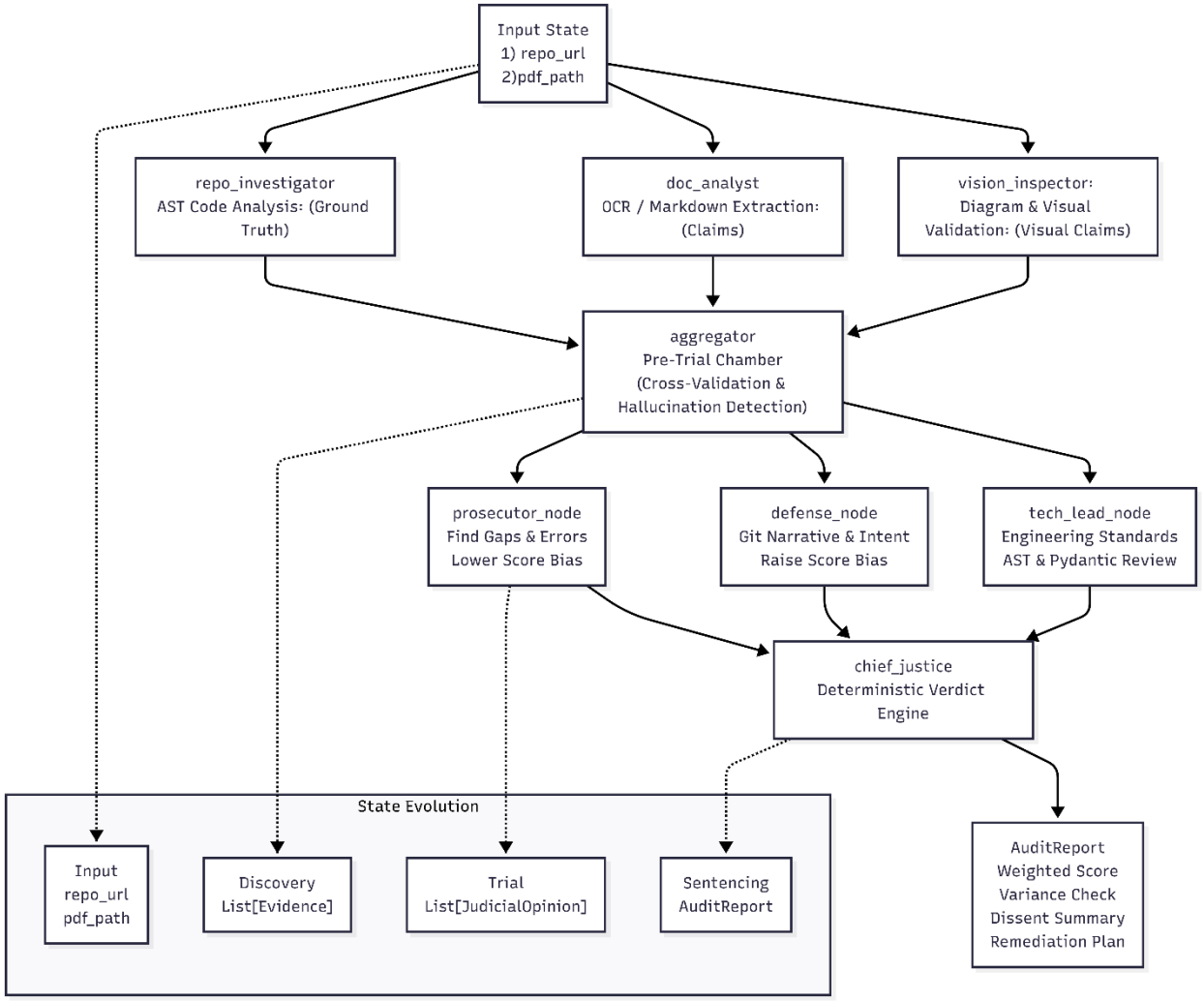
A known risk in parallel agents is **Opinion Convergence** (where agents start to sound the same). We mitigate this by:

1. **Isolated Contexts:** Each judge receives the evidence but *cannot* see the other judges' drafts.
2. **Structured Citations:** Each judge must link their **JudicialOpinion** to at least one Evidence object ID from the Detectives. An opinion without a citation is automatically discarded by the state reducer.

Tenacious Intelligence Corporation

4. Planned StateGraph Flow (Phase 1-4)

The full diagram that is expected by the end of this project looks like this:



- Solid lines = Control/Data flow between nodes
- Dotted lines = State evolution or state updates

Also, in simple terms:

- From Input State ⟶ Detectives: (repo_url, pdf_path)
- From Detectives ⟶ Aggregator: List[Evidence]
- From Aggregator ⟶ Judges: List[Evidence] or Cross-Validated Evidence
- From Judges ⟶ Chief Justice: List[JudicialOpinion]
- From Chief Justice ⟶ AuditReport: Final Verdict